R I G E L

Manual dos comandos ps e kill

Listar e encerrar tarefas (processos) no sistema operacional Rigel para 68000

Item	Informação
Documento	Manual de referência e guia de funcionamento interno
Comandos	`ps` (lista tarefas) e `kill` (encerra tarefas)
Plataforma	Rigel (kernel próprio, CPU 68000, tasks em slots de 256 KB)
Versão	1.0 — 23/10/2026
Arquivos principais	`sys_ctl.c`, `sys_ctl.h`, `scheduler.c`, `scheduler.h`, `ctx_switch_68000.S`, `trap0.c`, `loader_elf.c`, `ps.c`, `kill.c`

Sumário

1. Visão geral
2. Referência do comando ps
3. Referência do comando kill
4. Como funciona por dentro
5. Mapa dos arquivos e funções
6. Dando nome às tarefas
7. Diagnóstico e solução de problemas
8. Limitações conhecidas
9. Glossário

Este manual foi escrito para que você consiga entender, usar e modificar os dois comandos sem depender de ajuda externa. As seções 2 e 3 são referência de uso (como uma página de manual do Unix). As seções 4 a 8 explicam o código e o que fazer quando algo não funciona.

1. Visão geral

O Rigel executa várias tarefas ao mesmo tempo, alternando entre elas a cada tick do timer (10 ms). Duas ferramentas permitem observar e controlar essas tarefas:

- **ps** mostra uma linha por tarefa existente: número (PID), estado, tempo de CPU, endereço de memória e nome.
- **kill** encerra uma tarefa pelo PID, fechando os arquivos que ela deixou abertos e devolvendo a memória (o slot de 256 KB) ao sistema.

Os dois comandos são programas comuns (arquivos .elf carregados em um slot). Como um programa não enxerga a tabela de tarefas do kernel, cada um pede a informação ao kernel por uma chamada de sistema (syscall) usando `TRAP #0`.

1.1 Tipos de tarefa

Tipo	Como nasce	Pode ser morta pelo kill?
Tarefa do kernel	Criada na inicialização do kernel (por exemplo, as tarefas da rede). A pilha fica na memória do kernel.	Não. O kill responde “task do kernel, não pode ser morta”.
Programa (app)	Carregado de um .elf por `spawn`, em um slot de 256 KB. A pilha fica dentro do slot.	Sim.

O sistema decide o tipo olhando onde fica a pilha da tarefa: se ela está dentro de um slot, é um programa; se não, é do kernel. O número do PID não é usado para isso.

1.2 Como o PID é formado

O PID é o campo `id` da tarefa, o mesmo valor devolvido pela syscall `SYS_GETPID`. Para programas carregados pelo loader, o id é `OS_SLOT_ID_BASE + slot` (1000 + número do slot). Tarefas do kernel têm o id que quem as criou escolheu. Por isso você pode ver tarefas do kernel com PIDs altos (por exemplo 2000); isso é normal.

2. Referência do comando ps

NOME

`ps` — lista as tarefas existentes.

SINOPSE

```
ps
```

DESCRIÇÃO

O `ps` não recebe argumentos. Ele pede ao kernel uma fotografia da tabela de tarefas e imprime uma linha para cada tarefa existente. O próprio `ps` aparece na lista, no estado RUN, porque é a tarefa que está rodando no momento.

SAÍDA

```
 PID ESTADO    TICKS    BASE    NOME
2000 SLEEP       296  000000  w5100rx [kernel]
2001 SLEEP       769  000000  tcpip [kernel]
2002 SLEEP         5  000000  nm [kernel]
1000 SLEEP        41  0C2000  shell.elf
1001 RUN           3  102000  ps.elf
```

O exemplo acima é ilustrativo; os nomes e valores na sua máquina serão outros.

COLUNAS

Coluna	Significado
PID	Identificador da tarefa (campo `id`). É o número que se passa ao `kill`.
ESTADO	Situação atual da tarefa. Veja a tabela de estados abaixo.
TICKS	Tempo de CPU em ticks de 10 ms. Veja a observação sobre precisão abaixo.
BASE	Endereço inicial do slot de 256 KB da tarefa, em hexadecimal com 6 dígitos. Vale `000000` para tarefas do kernel, que não têm slot.
NOME	Nome da tarefa. A marca `[kernel]` indica tarefa do kernel. O traço – significa que a tarefa não recebeu nome (veja a seção 6).

ESTADOS

ESTADO	Nome interno	Significado
READY	`TASK_READY`	Pronta para rodar; está na fila do round robin esperando a vez.
RUN	`TASK_RUNNING`	Está usando a CPU neste instante. Em um sistema com um só processador, só uma tarefa por vez.
SLEEP	`TASK_BLOCKED` com `sleeping`	Dormindo por `OS_SleepTicks` ou `sleep`. Acorda sozinha quando o tempo termina. Não gasta CPU.
BLOCK	`TASK_BLOCKED` sem `sleeping`	Bloqueada esperando um evento externo (um driver vai acordá-la com `OS_TaskWake`).
KILL	`TASK_KILLING`	O kill está em andamento: a tarefa já saiu da fila e seus arquivos estão sendo fechados. Estado muito rápido.
DEAD	`TASK_DEAD`	Terminada, mas o slot ainda não foi devolvido. O `OS_Reap` recolhe. Como o `ps` chama o `OS_Reap` antes de listar, você raramente verá este estado.

OBSERVAÇÕES

- **Precisão do TICKS.** O contador é incrementado pela interrupção do timer na tarefa que está rodando naquele instante. É uma amostragem estatística, boa para comparar tarefas entre si, não uma medida exata. Tarefas que ficam quase sempre dormindo têm valores baixos.
- **Descobrir quem consome CPU.** Rode o `ps` duas vezes com alguns segundos de intervalo e compare a coluna TICKS. A tarefa cujo valor mais cresce é a que mais usa CPU.

- **Limite de linhas.** O kernel devolve no máximo `OS_MAX_TASKS` tarefas (12 na versão atual).
- **Efeito colateral.** Antes de listar, o `ps` recolhe tarefas mortas (`OS_Reap`), devolvendo seus slots.

CÓDIGO DE SAÍDA

0 em sucesso. 1 se a syscall falhar (é impressa a mensagem `ps: erro N`).

3. Referência do comando kill

NOME

`kill` — encerra tarefas pelo PID.

SINOPSE

```
kill [-SINAL] pid [pid ...]
```

DESCRIÇÃO

Encerra cada tarefa indicada. O comando aceita vários PIDs de uma vez e trata cada um de forma independente: se um falhar, os demais continuam sendo processados. Ao matar uma tarefa, o sistema fecha os arquivos, diretórios e sockets que ela deixou abertos e devolve o slot de memória.

SINAIS

O Rigel não tem tratadores de sinal como o Linux. Existe apenas “matar”. Os sinais aceitos são:

Sinal	Nome	Efeito
`-0`	SIG_CHECK	Não mata. Apenas verifica se o PID existe e se pode ser morto. Útil em scripts.
`-9`	SIGKILL	Mata a tarefa. É o padrão quando nenhum sinal é indicado.
`-15`	SIGTERM	Tratado exatamente como o `-9` (não existe encerramento “educado”).

Qualquer outro número entre 0 e 255 é aceito pelo comando, mas o kernel o recusa com erro -22 (inválido).

EXEMPLOS

```
kill 1001           mata a tarefa 1001
kill -9 1001        o mesmo (9 é o padrão)
kill -0 1001        só testa se a 1001 existe
kill 1001 1002      mata as duas
```

MENSAGENS

Mensagem	Quando aparece
`kill: (N) processo nao existe`	Não há tarefa viva com esse PID (erro -3, K_ESRCH).
`kill: (N) task do kernel, nao pode ser morta`	A tarefa é do kernel (erro -1, K_EPERM). Vale também para `kill -0`.
`kill: (N) erro -22`	Sinal inválido (diferente de 0, 9 e 15).
`kill: pid invalido: X`	O argumento não é um número positivo.
`kill: sinal invalido: -X`	O sinal não é um número de 0 a 255.
`uso: kill [-SINAL] pid [pid ...]`	Nenhum PID foi informado.

CÓDIGO DE SAÍDA

0 se todos os PIDs foram tratados sem erro. 1 se algum falhou ou se os argumentos eram inválidos.

PRECAUÇÕES

> **Atenção.** Se você matar o **shell** que está usando, o sistema continua funcionando, mas você fica sem prompt e não consegue digitar mais nada naquela sessão. Só outra sessão aberta ou um reset resolve. Use `ps` e confira o NOME antes de matar.

- Tarefas do kernel (rede, monitor etc.) não podem ser mortas por este comando. Isso é proposital, para não derrubar serviços do sistema por engano.
- Se o programa que você matou estava esperando algo de um driver, o driver pode ainda guardar uma referência a ele. O kernel já impede que a tarefa morta “ressuscite”, mas é um ponto a observar se aparecer comportamento estranho depois de um kill (seção 8).
- Matar a si mesmo (`kill` com o próprio PID) equivale a sair do programa: o comando não retorna.

4. Como funciona por dentro

4.1 O caminho de uma chamada

Um programa não pode ler a tabela de tarefas diretamente. O caminho completo é:

1. O programa (`ps.c` ou `kill.c`) chama `rigel_ps()` ou `rigel_kill()`, funções da librigel.
2. A função da librigel carrega o número da syscall em D0 e os argumentos nos registradores, e executa `TRAP #0`.
3. O processador entra no kernel e chama `trap0_handler()` (arquivo `trap0.c`).
4. O `switch` do `trap0_handler` reconhece `SYS_PS` (0x70) ou `SYS_KILL` (0x71) e chama `sys_ps()` ou `sys_kill()` (arquivo `sys_ctl.c`).
5. A função trabalha sobre a tabela de tarefas (no `scheduler.c`) e devolve um número. Esse número volta ao programa como valor de retorno.

> **Atenção.** Os números 0x70 e 0x71 são valores escolhidos em `sys_ctl.h`. Se o seu `sys/syscall.h` definir `SYS_PS` e `SYS_KILL`, valem os dele. Confira que não colidem com outras syscalls.

4.2 A estrutura TCB

Cada tarefa é descrita por um bloco `tcb_t` (Task Control Block), definido em `scheduler.h`. Os campos que interessam a estes comandos:

Campo	Função
`sp`	Ponteiro de pilha salvo. **Tem que ser o primeiro campo** (offset 0): o assembly de troca de contexto depende disso.
`state`	Estado da tarefa (READY, RUNNING, BLOCKED, DEAD, KILLING, UNUSED).
`id`	O PID.
`sleeping`	1 se estiver dormindo por tempo (`OS_SleepTicks`).
`stack_base`	Início da pilha. Usado para descobrir em qual slot a tarefa está.
`fds[]`	Tabela de descritores de arquivo da tarefa (32 entradas).
`ticks`	Contador de ticks de CPU (novo, usado pelo `ps`).
`name[32]`	Nome da tarefa (novo, usado pelo `ps`).

Os campos novos `ticks` e `name` foram colocados no final da estrutura, depois de `fds`, para não mexer no offset de `sp`.

4.3 Como o ps obtém os dados

A função `sys_ps(buf, max)` (em `sys_ctl.c`) faz, nesta ordem:

1. Valida o buffer: não pode ser nulo nem ter endereço ímpar (o 68000 gera address error em acesso a palavra em endereço ímpar). Limita `max` a `OS_MAX_TASKS`.
2. Chama `OS_Reap()` para recolher tarefas mortas.
3. Desabilita as interrupções (`irq_save`) para tirar uma fotografia consistente da tabela.
4. Percorre as 12 posições do pool. Para cada tarefa que não está UNUSED, preenche uma entrada `task_info_t` com PID, ticks, estado, flags, BASE e nome.
5. Reabilita as interrupções e devolve a quantidade de entradas.

A coluna BASE vem de `OS_TaskSlot(t)`, que calcula o número do slot a partir da pilha: `(stack_base - SLOT_BASE) / SLOT_SIZE`. Se a pilha está fora da área dos slots, o resultado é -1 e a tarefa é marcada como do kernel.

4.4 Como o contador TICKS anda

A cada 10 ms ocorre a interrupção do timer e roda `OS_TickISR` (em `ctx_switch_68000.S`). Depois de salvar o contexto da tarefa interrompida, ela chama a função C `OS_AccountTick()`, que faz `current_task->ticks++`, e em seguida chama `OS_Schedule()` para escolher a próxima tarefa. Os registradores já estão salvos pelo `movem.l`, por isso chamar uma função C ali é seguro.

4.5 Como o kill funciona

A função `sys_kill(pid, sinal)` segue este roteiro:

Passo	O que faz	Por quê
1	Valida o PID e o sinal.	Recusa PID 0, PID negativo ou sinal diferente de 0, 9 e 15 (erro -22).
2	Procura a tarefa viva com esse PID (`find_task`).	Se não achar: erro -3 (não existe).
3	Verifica se é do kernel (`OS_TaskSlot < 0`).	Se for: erro -1 (não permitido). Isso vale também para o sinal 0.
4	Se o sinal é 0, devolve 0.	O sinal 0 só testa a existência.
5	Se a tarefa é a própria que chamou, fecha recursos e chama `OS_TaskExit()`.	Mesmo caminho do `SYS_EXIT`. Não volta.
6	`OS_TaskKillBegin`: tira a tarefa da fila do round robin e a marca KILLING.	A partir daqui ela não recebe mais CPU. Ninguém a acorda e ninguém reaproveita o TCB.
7	`close_task_resources`: chama `sys_fs_task_exit`, `sys_dir_task_exit` e `sys_net_task_exit`.	Fecha arquivos FAT, diretórios e sockets da tarefa (esses módulos guardam os dados por id).
8	`vfs_task_exit`: fecha todos os descritores da tabela `fds`.	Evita vazamento de `File` e avisa a outra ponta de um pty ou socket.
9	`OS_TaskKillEnd`: marca a tarefa como DEAD.	Agora ela pode ser recolhida.
10	`OS_Reap`: devolve o slot (`Slots_Free`) e o TCB (UNUSED).	A memória volta a ficar disponível para novos programas.

Por que existe o estado KILLING

Entre tirar a tarefa da fila e terminar de fechar os arquivos, o sistema precisa garantir que ninguém use o TCB. Se a tarefa fosse marcada DEAD logo no início, um `spawn` simultâneo poderia chamar `OS_Reap`, entregar o mesmo TCB a um programa novo e depois o `kill` fecharia os arquivos do programa novo. O estado KILLING evita isso: não é UNUSED (ninguém aloca), não é DEAD (ninguém recolhe) e não é BLOCKED (ninguém acorda).

Por que a tarefa morta não ressuscita

`OS_TaskWake` e `wake_sleepers` só acordam tarefas em estado BLOCKED. Uma tarefa em KILLING ou DEAD é ignorada, mesmo que um driver ainda tente acordá-la. Qualquer código novo que acorde tarefas deve manter essa regra: **só acordar se o estado for BLOCKED**.

4.6 Ciclo de vida de uma tarefa

De	Para	Quem causa
UNUSED	READY	`OS_TaskCreate` (spawn ou criação no boot).
READY	RUNNING	`OS_Schedule` escolhe a tarefa.
RUNNING	READY	Fim da fatia de tempo (tick) ou `OS_Yield`.
RUNNING / READY	BLOCKED	`OS_TaskBlock` ou `OS_SleepTicks`.
BLOCKED	READY	`OS_TaskWake` ou `wake_sleepers` (fim do sono).
READY / RUNNING / BLOCKED	KILLING	`OS_TaskKillBegin` (kill de outra tarefa).
KILLING	DEAD	`OS_TaskKillEnd`.
RUNNING	DEAD	`OS_TaskExit` (saída normal ou kill de si mesma).
DEAD	UNUSED	`OS_Reap` (também devolve o slot).

5. Mapa dos arquivos e funções

Arquivo	Conteúdo relacionado a ps e kill
`sys_ctl.h`	Tipos e constantes compartilhados: `task_info_t`, estados `PS_*`, flags, sinais, códigos de erro `K_*`, números `SYS_PS` e `SYS_KILL`, protótipos.
`sys_ctl.c`	As syscalls `sys_ps` e `sys_kill`, mais `find_task` e `close_task_resources`.
`scheduler.h` / `scheduler.c`	O `tcb_t`, o estado `TASK_KILLING`, `OS_TaskSetName`, `OS_AccountTick`, `OS_TaskAt`, `OS_TaskSlot`, `OS_TaskKillBegin`, `OS_TaskKillEnd`; `OS_Reap` usa `OS_TaskSlot`.
`ctx_switch_68000.S`	`OS_TickISR` chama `OS_AccountTick` a cada tick.
`trap0.c`	Os dois `case`: `SYS_PS` e `SYS_KILL`.
`loader_elf.c`	Cria a tarefa de cada programa e (se você aplicou a mudança da seção 6) chama `OS_TaskSetName`.
`vfs.c`	`vfs_task_exit`, que fecha todos os descritores de uma tarefa.
`slots.c` / `slots.h`	Os slots de 256 KB: `Slots_Alloc`, `Slots_Free`, `Slots_BaseAddr`. 10 slots a partir de 0x0C2000.
`ps.c`, `kill.c`	Os programas de usuário.
librigel	`rigel_ps` e `rigel_kill`, que executam o `TRAP #0`.

6. Dando nome às tarefas

O nome é gravado no TCB pela função `OS_TaskSetName(tcb, "nome")`. O nome é cortado em 31 caracteres. O nome fica vazio quando a tarefa nasce, e o `ps` mostra `-` enquanto ninguém o define. Quando um TCB é devolvido ao pool, o `OS_Reap` apaga o nome e zera os ticks.

6.1 Programas carregados pelo loader

No `loader_elf.c`, logo depois de criar a tarefa com `OS_TaskCreate` e testar `if (!tcb)`, acrescente:

```
{
    const char *nm = (argc > 0 && argv[0]) ? argv[0] : "?";
    const char *p;
    for (p = nm; *p; p++)
        if (*p == '/')
            nm = p + 1;          /* "/bin/ls.elf" -> "ls.elf" */
    OS_TaskSetName(tcb, nm);
}
```

O laço existe porque o kernel não tem `strrchr`. Ele guarda a posição seguinte a cada barra, e no fim `nm` aponta para o nome do arquivo.

6.2 Tarefas criadas no boot

Onde o kernel cria as tarefas estáticas, guarde o retorno e dê o nome:

```
tcb_t *t = OS_TaskCreate(entrada, NULL, pilha, tamanho, id);
OS_TaskSetName(t, "w5100rx");
```

> **Dica.** Para saber qual tarefa é qual sem alterar o código, compare o PID mostrado pelo `ps` com o `id` passado a cada `OS_TaskCreate` no código do kernel.

7. Diagnóstico e solução de problemas

Sintoma	Causa provável	O que fazer
BASE com endereço que não existe na máquina (por exemplo FAC2000).	Versão antiga, que deduzia o slot pelo id. Tarefas do kernel com id alto eram tratadas como programas.	Use a versão atual: o slot vem da pilha (`OS_TaskSlot`). Tarefa do kernel mostra BASE 000000.
NOME aparece como `-`.	Ninguém chamou `OS_TaskSetName` para essa tarefa.	Veja a seção 6.
TICKS não aumenta.	A linha do `OS_TickISR` que chama `OS_AccountTick` não está no assembly, ou a tarefa quase só dorme.	Confira o `ctx_switch_68000.S`. Compare duas execuções do `ps`.
Depois do kill o sistema ficou sem prompt.	O PID morto era o do shell que você usa.	Reset ou outra sessão. Confira o NOME antes de matar.

Sintoma	Causa provável	O que fazer
`kill` diz “processo nao existe” para PID que o ps mostra.	A tarefa está em KILL ou DEAD (já sendo encerrada), ou o PID foi digitado errado.	Rode `ps` de novo.
Slots acabam depois de muitos programas.	Os slots só voltam em `OS_Reap`. Verifique que o `OS_Reap` usa `OS_TaskSlot` e não o id.	Rode `ps` (ele recolhe) e confira o BASE dos novos programas.
Travamento durante o kill.	Algum módulo de fechamento (arquivo, diretório, rede ou vfs) está esperando algo.	Use os marcadores da seção 7.1.

7.1 Marcadores de depuração do kill

No `sys_ctl.c` existe a linha `#define KILL_DEBUG 1`. Com ela, cada etapa do kill imprime uma linha na tela do kernel:

```
[kill] 1 procura
[kill] 2 tira da fila
[kill] 3 fs/dir/net
[kill] 4 fds (vfs_task_exit)
[kill] 5 marca DEAD
[kill] 6 reap
[kill] 7 fim
```

A última linha que aparecer indica onde o sistema parou. Por exemplo, se a última foi a 3, o problema está em `sys_fs_task_exit`, `sys_dir_task_exit` ou `sys_net_task_exit`. Quando terminar de depurar, comente a linha `#define KILL_DEBUG 1` e as mensagens somem.

7.2 Como testar com segurança

1. Rode `ps` e anote os PIDs e nomes.
2. Abra um programa simples (que só dorme e não abre arquivo nem conexão) e use `kill` nele.
3. Rode `ps` outra vez: o PID deve ter sumido, e o slot deve estar livre para o próximo programa.
4. Só depois teste em programas que usam arquivos, rede ou telnet.

8. Limitações conhecidas

- **Um só tipo de sinal.** Não existe tratador de sinal nem encerramento “educado”. `SIGTERM` equivale a `SIGKILL`.
- **Sem controle de permissão.** Qualquer programa pode matar qualquer outro programa. Só as tarefas do kernel são protegidas.
- **Tarefa morta e drivers.** Se um driver guardou o ponteiro do TCB de uma tarefa que estava bloqueada nele, o ponteiro pode apontar para um TCB reaproveitado depois. O `OS_TaskWake` só acorda tarefas BLOCKED, mas um novo programa que esteja bloqueado no mesmo TCB poderia ser acordado fora de hora. Se perceber isso, os drivers devem apagar essas referências no fechamento do fd.

- **Módulos de limpeza.** `sys_fs_task_exit`, `sys_dir_task_exit` e `sys_net_task_exit` foram escritos para rodar dentro da própria tarefa que sai. No kill de outra tarefa eles são chamados por uma tarefa diferente; confirme que não usam `current_task` por dentro.
- **Máximo de 12 tarefas** (`OS_MAX_TASKS`) e 10 slots de 256 KB.
- **TICKS é por amostragem**, não uma medida exata.
- **Interrupções desabilitadas.** Em operações de disco o kernel desabilita interrupções por decisão do projeto; isso reduz, mas não elimina, a chance de matar uma tarefa no meio de uma operação sensível.

9. Glossário

Termo	Definição
Tarefa (task)	Unidade de execução do Rigel. O mesmo que processo, neste manual.
PID	Número que identifica a tarefa. No Rigel é o campo `id` do TCB.
TCB	Task Control Block. Estrutura com os dados de uma tarefa: pilha salva, estado, id, arquivos abertos.
Slot	Bloco de 256 KB de memória reservado a um programa. Tem o código, os dados e a pilha. Existem 10, a partir de 0x0C2000.
Tick	Interrupção do timer a cada 10 ms. Marca o ritmo do escalonador.
Round robin	Política em que as tarefas prontas se revezam na CPU em ordem circular.
Syscall	Chamada de sistema: pedido de um programa ao kernel, feito com `TRAP #0`.
fd	File descriptor: número que identifica um arquivo, dispositivo ou socket aberto.
Reap	Recolher uma tarefa terminada: liberar o slot e devolver o TCB ao pool.
Irq mascarada	Interrupções desabilitadas por um instante, para mexer em estruturas compartilhadas sem que o tick interrompa no meio.
Código de retorno K_*	`K_EPERM` (-1) não permitido; `K_ESRCH` (-3) não existe; `K_EINVAL` (-22) argumento inválido.